CSV Files in Java

19 October 2024 18:30

A CSV(Comma Separated Values) file is a simple text format for storing tabular data, such as a spreadsheet or database, where each line represents a row, and each value in a row is separated by comma. CSV files are widely used because of their simplicity and ease of integration with different Systems.

In Java, handling csv files involves reading from and writing to these files using standard file I/O Operations. you can use several libraries and built in classes for this task, such as:

- java.io package(built-in)
- OpenCSV (third-party library)
- Apache Commons CSV (third-party library)

Structure of a CSV File

ACSV file can. file consist of rows of data where each value is separated by a comma. Each row represents a record, and the values in the row represents the field of that record.

CSV File:

```
empId,name,age,city,basicSal
e001,Jhon,35,New York,4500
e002,Amber,35,New York,4500
e003,Eva, 33,Madison,4200
e004,Joe, 32,Madison,4250
e005,Racheal,33,Chicago,4600
```

← Header row

← Data rows

Reading a CSV File in Java (Using java.io package)

BufferedReader and FileReader from the java.io package. The idea is to read the file line by line and split each line into its fields using String.split() based on commas.

```
package FileHandling1.src;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;
public class ReadCSV {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String filePath = "./FileHandling1/CSVFiles/";
        String line;
        String delimiter=",";
        System.out.println("Enter your csv file name: ");
        String fileName = sc.nextLine();
        try (BufferedReader br = new BufferedReader(new FileReader(filePath+fileName))) {
            //read the header
            String header = br.readLine();
            System.out.println("header: "+header);
            //Read and process each line of the CSV
            while((line = br.readLine())!= null){
                //Split each line into fields using delimiter
                String[] fields = line.split(delimiter);
                //Accessing individual fields
                String eCode = fields[0];
```

```
                String eName = fields[1];
                String eAge = fields[2];
                String eCity = fields[3];
                String eSalary = fields[4];
                System.out.println(
                    "\nEmployee code:" + eCode +
                    "\nName: " + eName +
                    "\nAge: " + eAge +
                    "\nCity: " + eCity +
                    "\nSalary: " + eSalary
                );
            }
        } catch (IOException e) {
            // TODO: handle exception
            e.printStackTrace();
        }
        finally{
            sc.close();
        }
    }
}
```

Output:

```
Enter your csv file name:
one.csv
header: empId,name,age,city,basicSal

Employee code:e001
Name: Jhon
Age: 35
City: New York
Salary: 4500

Employee code:e002
Name: Amber
Age: 35
City: New York
Salary: 4500

Employee code:e003
Name: Eva
Age:  33
City: Madison
Salary: 4200

Employee code:e004
Name: Joe
Age:  32
City: Madison
Salary: 4250

Employee code:e005
Name: Racheal
Age: 33
City: Chicago
Salary: 4600
```

Writing to a CSV File in Java(Using java.io package)

Writing to a csv file is similar to writing to a text file. You can use FileWriter and BufferedWriter to write data in a csv format with comma separating the fields.

```
package FileHandling1.src;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;
public class ReadCSV {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String filePath = "./FileHandling1/CSVFiles/";
        String line;
        String delimiter=",";
        System.out.println("Enter your csv file name: ");
        String fileName = sc.nextLine();
        try (BufferedReader br = new BufferedReader(new FileReader(filePath+fileName))) {
            //read the header
            String header = br.readLine();
            System.out.println("header: "+header);
            //Read and process each line of the CSV
            while((line = br.readLine())!= null){
                //Split each line into fields using delimiter
                String[] fields = line.split(delimiter);
                //Accessing individual fields
                String eCode = fields[0];
                String eName = fields[1];
                String eAge = fields[2];
                String eCity = fields[3];
                String eSalary = fields[4];
                System.out.println(
                    "\nEmployee code:" + eCode +
                    "\nName: " + eName +
                    "\nAge: " + eAge +
                    "\nCity: " + eCity +
                    "\nSalary: " + eSalary
                );
            }
        } catch (IOException e) {
            // TODO: handle exception
            e.printStackTrace();
        }
        finally{
            sc.close();
        }
    }
}
```

CRUD stands for Create, Read, Update, and Delete, which are the four fundamental operations for managing data in a database or application:

- **Create**: Allows users to add new records to a database
- **Read**: Allows users to search for and retrieve specific records from a database
- **Update**: Allows users to modify existing records in a database
- **Delete**: Allows users to remove records from a database

- **Read**: Allows users to search for and retrieve specific records from a database
- **Update**: Allows users to modify existing records in a database
- **Delete**: Allows users to remove records from a database

RollNum	Name	term 1	term 2
S001	XYZ	-	-
S002	YZX	-	-
S003	ZYX	-	-
S004	YXZ	-	-

Student.csv

Write programs to perform CRUD operation on Student.csv. Your source files and csv files must be stored in the seperate folder.